UNIVERSIDAD CHAMPAGNAT

Licenciatura en Sistemas de Información
Programación Distribuida
Trabajo Práctico 4
Consenso Distribuido, Paxos y Raft

FECHA:

APELLIDO Y NOMBRE:

DNI:

UNIVERSIDAD CHAMPAGNAT

CONSIGNAS

1. Objetivo

El objetivo del presente Trabajo Práctico es continuar la evolución de la aplicación desarrollada en el Trabajo Práctico N° 3 e introducir, mediante una experiencia de laboratorio, problemas propios de los sistemas distribuidos con múltiples réplicas: elección de un coordinador, acuerdo por mayoría, replicación ordenada de operaciones, tolerancia a fallos y recuperación del estado.

La parte práctica se centrará en una implementación educativa y simplificada de Raft sobre un cluster fijo de tres nodos. Paxos se trabajará de manera conceptual y comparativa; no se requiere implementarlo en código.

2. Forma de trabajo: desarrollo por hitos

El TP se organiza en hitos secuenciales. Cada hito resuelve un problema concreto y deja un comportamiento observable que será utilizado por el siguiente. La intención es evitar una implementación monolítica de Raft y facilitar la detección de errores durante el desarrollo.

En los puntos de mayor complejidad se incluye una guía o abstracción de posible implementación. Estas ayudas describen responsabilidades, estados o secuencias, pero no constituyen código resuelto ni obligan a utilizar una estructura exacta.

3. Punto de partida y continuidad

El trabajo deberá partir del proyecto entregado en el TP N° 3. Se deberán conservar, en la medida en que resulten compatibles con la nueva arquitectura, la API REST, HTTPS, MongoDB/Mongoose, logging con Winston, documentación OpenAPI/Swagger y configuración mediante variables de entorno.

Redis/Memurai podrá conservarse para las funcionalidades de concurrencia heredadas del TP N° 2, pero no deberá utilizarse para elegir Leader, almacenar votos, calcular quorum, decidir commit ni reemplazar la comunicación propia del protocolo Raft.

4. Alcance y simplificaciones permitidas

- Cluster fijo de tres nodos ejecutados como procesos independientes.
- Comunicación interna entre nodos mediante HTTP en el entorno local.
- Fallas consideradas de tipo crash/restart: un nodo puede detenerse y volver a iniciarse.
- Cada nodo deberá utilizar una base MongoDB o espacio de persistencia claramente independiente.
- No se exige tolerancia a fallas bizantinas, membresía dinámica, snapshots, log compaction, ReadIndex, PreVote, Joint Consensus ni optimizaciones de producción.
- La API externa podrá conservar HTTPS; la comunicación interna del protocolo puede utilizar HTTP para simplificar el laboratorio.
- Las lecturas GET pueden resolverse localmente. El foco de consenso del TP estará puesto en las operaciones que modifican estado.

5. Secuencia obligatoria de hitos

Hito	Problema principal	Resultado observable	Habilita
1	Cluster e identidad	Tres réplicas independientes y observables	Hito 2
2	Elección y quorum	Leader elegido por mayoría	Hito 3
3	Heartbeats y failover	Liderazgo estable y reemplazable	Hito 4

Hito	Problema principal	Resultado observable	Habilita
4	Log replicado y commit	Entradas replicadas y confirmadas	Hito 5
5	State Machine y API REST	Estado de negocio convergente	Hito 6
6	Recuperación y persistencia	Reinicio y follower atrasado recuperables	Cierre

PARTE I - CLUSTER, IDENTIDAD Y ELECCIÓN

HITO 1 - Cluster de tres nodos y estado observable

Problema a resolver

Antes de implementar consenso, cada proceso debe saber quién es y con quién puede comunicarse. Las tres réplicas deben poder ejecutarse simultáneamente sin compartir de manera accidental el mismo estado de negocio.

Tareas

- Adaptar el proyecto para iniciar tres instancias independientes de la misma aplicación.
- Asignar a cada instancia un identificador de nodo y direcciones/puertos que permitan distinguir su API externa de la comunicación interna.
- Utilizar persistencia de negocio separada para cada nodo.
- Crear una vista o endpoint de diagnóstico que permita observar como mínimo identidad del nodo, rol, término actual y Leader conocido.
- Documentar cómo se inicia cada instancia sin editar el código fuente entre ejecuciones.

GUÍA / ABSTRACCIÓN: Una posible organización consiste en mantener una configuración común del cluster y elegir la identidad local al iniciar el proceso. A partir de esa identidad pueden derivarse puertos, base de datos y lista de peers. No es obligatorio usar esta estructura.

```
proceso + nodeId
      |
      +--> configuración local (API / Raft / Mongo)
      +--> lista de peers
      +--> estado observable del nodo
```

Evidencia mínima

- Tres procesos ejecutándose simultáneamente.
- Tres identidades distintas.
- Tres espacios de persistencia independientes.
- Consulta del estado de cada nodo.

HITO 2 - Elección de Leader mediante mayoría

Problema a resolver

El cluster no debe depender de un Leader configurado manualmente. Los nodos deberán decidir cuál de ellos coordina el término actual utilizando votos y mayoría.

Tareas

- Representar los roles FOLLOWER, CANDIDATE y LEADER.
- Mantener un número de término y el voto emitido dentro del término actual.
- Incorporar un election timeout no idéntico para todos los nodos.

- Al vencer el timeout, permitir que el nodo inicie una elección, incremente su término y solicite votos a los peers.
- Implementar un RPC de solicitud de voto y las validaciones necesarias para decidir si el voto puede concederse.
- Calcular la mayoría en función del tamaño del cluster y declarar Leader únicamente al alcanzar quorum.

GUÍA / ABSTRACCIÓN: Pensar la elección como una máquina de estados. El timeout dispara la transición FOLLOWER -> CANDIDATE. El candidato incrementa el término, registra su propio voto y consulta a los peers. Las respuestas pueden otorgar el voto, rechazarlo o informar un término superior.

```
timeout
  -> CANDIDATE
  -> term = term + 1
  -> voto propio
  -> solicitar votos
  -> ¿mayoría? -> LEADER
  -> ¿término superior? -> FOLLOWER
```

Preguntas de control

1. ¿Por qué un nodo no debería votar por dos candidatos diferentes dentro del mismo término?
2. ¿Qué debería ocurrir si un mensaje recibido contiene un término mayor al término local?
3. ¿Puede un único nodo de un cluster de tres declarar por sí solo que existe consenso?

PARTE II - ESTABILIDAD DEL LIDERAZGO Y REPLICACIÓN

HITO 3 - Heartbeats y failover

Problema a resolver

Una elección aislada no alcanza: si los followers no reciben señales del Leader, volverán a iniciar elecciones. Además, el cluster debe reaccionar si el Leader realmente deja de estar disponible.

Tareas

- Implementar mensajes periódicos del Leader hacia los followers.
- Utilizar esos mensajes para reiniciar el mecanismo de detección de ausencia de Leader.
- Evitar elecciones innecesarias mientras exista un Leader válido.
- Detener el envío de heartbeats cuando el nodo deja de ser Leader.
- Forzar la caída del Leader y verificar que los followers puedan realizar una nueva elección.
- Reincorporar el nodo caído y verificar que adopte el término y liderazgo vigentes cuando corresponda.

GUÍA / ABSTRACCIÓN: Raft utiliza AppendEntries tanto para replicar entradas como para enviar heartbeats. En esta etapa puede comenzarse con un mensaje sin entradas. Un follower debería renovar su election timer sólo cuando recibe un mensaje válido del Leader para un término aceptable.

```
LEADER -- mensaje periódico --> FOLLOWER
                              -> reconoce Leader
                              -> actualiza término si corresponde
                              -> reinicia election timer
```

HITO 4 - Log replicado, consistencia y commit

Problema a resolver

El Leader debe poder proponer operaciones y lograr que las réplicas acuerden un orden común. Una operación no puede considerarse confirmada simplemente porque fue recibida o escrita localmente por el Leader.

Tareas

- Definir una estructura de entrada de log que permita identificar su posición, término y operación.
- Permitir que el Leader agregue una operación a su log local y la envíe a los followers.
- Hacer que el follower verifique que la nueva entrada es compatible con la parte previa de su log antes de aceptarla.
- Registrar en el Leader el progreso conocido de replicación de cada follower.
- Definir cuándo una entrada puede considerarse committed de acuerdo con la mayoría del cluster.
- Propagar a los followers el avance del commit para que todos puedan reconocer las entradas confirmadas.

GUÍA / ABSTRACCIÓN: Una entrada puede modelarse con tres piezas mínimas: índice, término y comando. Para comprobar continuidad, el mensaje de replicación puede incluir una referencia a la entrada inmediatamente anterior. El Leader necesita además alguna forma de recordar hasta dónde llegó cada follower.

```
entrada = { posición, término, comando }

Leader:
  agrega entrada local
  -> envía desde la posición que espera cada follower
  -> actualiza progreso según respuesta
  -> si una mayoría contiene la entrada, puede avanzar commit
```

PISTA PARA FOLLOWERS ATRASADOS: Si un follower informa que no puede enlazar la nueva entrada con su log, el Leader necesita una estrategia de reintento desde una posición anterior. En esta etapa basta con diseñar un mecanismo correcto para el cluster fijo; no se exige optimización.

Pruebas requeridas

- Con los tres nodos activos, proponer una operación y demostrar que aparece en los tres logs.
- Detener un follower y demostrar que Leader + follower restante pueden alcanzar mayoría.
- Dejar solamente un nodo activo e intentar una nueva operación. Analizar por qué no debería confirmarse.
- Distinguir en la evidencia entre entrada local, entrada replicada y entrada committed.

PARTE III - STATE MACHINE E INTEGRACIÓN CON LA API

HITO 5 - State Machine replicada e integración del CRUD

Problema a resolver

El log acordado debe producir un estado de negocio consistente. La base MongoDB no debe modificarse como paso previo al consenso y luego "avisar" a los demás nodos; las entradas confirmadas deben ser la fuente de las modificaciones de estado.

Tareas

- Implementar una State Machine que procese las entradas committed respetando el orden del log.

- Mantener un indicador del último índice aplicado para evitar saltos o desorden.
- Representar las operaciones de negocio como comandos suficientemente completos para producir el mismo resultado en cada réplica.
- Integrar al menos POST, PUT y DELETE del recurso del TP3 con el mecanismo de consenso.
- Definir y documentar qué ocurre cuando un cliente envía una escritura a un follower.
- Comprobar que las bases locales convergen después de aplicar las mismas entradas committed.

GUÍA / ABSTRACCIÓN: Separar dos responsabilidades: el protocolo decide qué entrada está committed; la State Machine aplica esa decisión al modelo de negocio. Conviene que el comando contenga toda información no determinística que deba ser igual en las réplicas, por ejemplo identificadores o marcas de tiempo generadas una sola vez.

```
Cliente -> API del Leader
       -> comando
       -> log Raft
       -> quorum / commit
       -> State Machine
       -> MongoDB local de cada nodo
```

DECISIÓN DE DISEÑO: Una escritura recibida por un follower no debe aplicarse localmente. El estudiante debe definir una conducta explícita: rechazo informando el Leader, redirección o proxy controlado. La alternativa elegida debe estar documentada y ser consistente.

Validaciones sugeridas

- Crear un recurso desde el Leader y verificar el mismo identificador en todas las bases.
- Actualizar el recurso y verificar la misma versión del dato en las réplicas.
- Eliminar el recurso y comprobar convergencia.
- Enviar una escritura deliberadamente a un follower y documentar la respuesta elegida.
- Verificar que una entrada no committed no sea aplicada como estado definitivo.

PARTE IV - RECUPERACIÓN, PERSISTENCIA Y CIERRE

HITO 6 - Recuperación de followers y persistencia del estado Raft

Problema a resolver

Un sistema distribuido debe continuar siendo coherente cuando un follower queda atrasado o cuando los procesos se reinician. El estado crítico del protocolo no puede depender exclusivamente de la memoria del proceso.

Parte A - Follower atrasado

- Detener un follower.
- Confirmar una o más operaciones utilizando los dos nodos restantes.
- Reiniciar el follower atrasado.
- Detectar la diferencia entre su log y el del Leader.
- Recuperar las entradas faltantes hasta alcanzar convergencia.

GUÍA / ABSTRACCIÓN: El Leader puede mantener, para cada follower, una posición desde la cual intentará la siguiente replicación y una posición hasta la cual sabe que el follower coincide. Si una replicación falla por falta de continuidad, se puede retroceder el punto de envío y reintentar.

```
Leader conoce progreso del follower
  -> intenta AppendEntries desde posición N
  -> rechazo por inconsistencia
  -> ajusta N
  -> reintenta
  -> follower alcanza el log del Leader
```

Parte B - Persistencia

Determinar qué información del protocolo debe sobrevivir a un reinicio y persistirla en un almacenamiento local del nodo.

GUÍA / ABSTRACCIÓN: Como mínimo deben analizarse el término actual, el voto emitido y el log. También puede persistirse el progreso de commit/aplicación para simplificar la recuperación. En cambio, rol actual, Leader conocido, índices de replicación del Leader y timers pueden tratarse como estado reconstruible.

Prueba de reinicio completo

1. Generar varias entradas y confirmar que fueron committed.
2. Registrar el estado de los tres nodos.
3. Detener completamente los tres procesos.
4. Reiniciar el cluster sin borrar las bases.
5. Esperar una nueva elección.
6. Comprobar que el término no retrocede y que el log y estado confirmado continúan disponibles.

6.1. Actividad de integración con TP2 - Redis/Memurai y Raft

Una vez completados los seis hitos, se propone integrar una operación concurrente heredada del TP2 para analizar cómo se complementan exclusión mutua y consenso. Esta actividad se realiza al final para no mezclar ambos problemas durante la construcción básica de Raft.

Tomar una operación de actualización concurrente sobre un recurso compartido -por ejemplo una reserva o decremento de stock- y proteger la sección crítica con el lock distribuido ya desarrollado. La modificación aceptada deberá ingresar luego al mecanismo Raft para ser replicada y confirmada.

ABSTRACCIÓN: Redis/Memurai decide quién puede entrar a la sección crítica en ese instante. Raft decide cómo se ordena, replica y confirma la modificación aceptada. Redis no debe utilizarse como servicio de elección o commit.

7. Observabilidad y documentación

La implementación deberá permitir comprender qué está ocurriendo en el cluster sin depender únicamente de depuración paso a paso. Se deberán conservar y ampliar logging y Swagger/OpenAPI.

- Registrar eventos significativos de elección, cambio de término, liderazgo, heartbeats, replicación, commit, aplicación y recuperación.
- Exponer endpoints de diagnóstico para consultar el estado del nodo y el log.
- Documentar la API de negocio y diferenciarla de los RPC internos del protocolo.
- Evitar utilizar manualmente RPC internos que modifican el protocolo como mecanismo ordinario de prueba de negocio.

8. Pruebas finales obligatorias

Al finalizar el trabajo, la demostración deberá recorrer los siguientes escenarios sin modificar el código entre pruebas:

Escenario	Acción	Comportamiento a demostrar
A	Arranque de tres nodos	Se identifica un Leader y dos followers.
B	Escritura sobre follower	La operación no se aplica localmente saltándose el consenso.
C	CRUD sobre Leader	Las operaciones confirmadas convergen en los tres nodos.
D	Caída del Leader	Los nodos restantes realizan una nueva elección.
E	Quorum 2/3	El cluster puede confirmar nuevas operaciones.
F	Sin quorum 1/3	No se confirman nuevas escrituras.
G	Follower atrasado	Al regresar, recupera las entradas faltantes.
H	Restart completo	Se recuperan estado crítico y entradas confirmadas.
I	Integración Redis + Raft	Dos operaciones concurrentes muestran exclusión mutua y una modificación confirmada.

9. Entrega

La entrega es individual y deberá permitir reproducir el funcionamiento del cluster a partir de la documentación presentada.

Incluir

- Código fuente.
- package.json y package-lock.json.
- .env.example completo y sin secretos.
- README.md con instalación, configuración, ejecución de los tres nodos y secuencia de pruebas.
- Swagger/OpenAPI actualizado.
- Evidencias seleccionadas de los hitos y pruebas finales.

No incluir

- node_modules/.
- .env real.
- Contraseñas, tokens o credenciales.
- Claves privadas o certificados PFX reales.
- Logs voluminosos que no sean necesarios como evidencia.

10. Consideraciones finales

El objetivo del TP no es copiar una implementación completa de Raft, sino poder explicar y demostrar cómo un conjunto de réplicas independientes acuerda liderazgo, orden de operaciones y confirmación por mayoría frente a fallas controladas.